

מבחן במבוא למדעי המחשב – 67101

מועד א' 31.1.18

מורי הקורס: אביב זהר וג'ף רוזנשיין

במבחן זה שני חלקים. יש לענות על כל 10 השאלות בחלק א', ולבחור 4 מתוך 6 השאלות בחלק ב' (אין לענות על יותר מ-4 שאלות בחלק ב').

יש לענות על כל השאלות בגוף השאלון – בתוך "קופסאות התשובה" בלבד. אין להגיש דפי טיוטא – הם לא יבדקו.

חומר מותר לשימוש בבחינה: אין.
משך הבחינה: 3 שעות.

סמנו כאן על אילו שאלות בחרתם לענות בחלק ב :

6	5	4	3	2	1
---	---	---	---	---	---

(הקיפו בעיגול)

חלק א' – שאלות סגורות (40% -- יש לענות על כל 10 הסעיפים, כל סעיף מזכה ב-4 נק')

בחלק זה של המבחן יש לכתוב את התשובה הסופית בלבד בתוך המשבצת המתאימה. אין צורך לנמק, ונימוקים גם לא יקראו.

1. מה תדפיס התוכנית הבאה?

```
def foo(data, le):  
    result = []  
    while le:  
        result.append(data[:le[0]])  
        data = data[le[0]:]  
        le = le[1:]  
    return result
```

```
print(foo(list(range(10)), [2, 2, 4]))
```

--

2. מה תדפיס התוכנית הבאה?

```
def square_it(var): return [var*2]*2

zoo = square_it([1])
zoo[1][0] += 1
print(zoo)
```

3. מה תדפיס התוכנית הבאה?

```
def c1(func):
    def outer(arg1):
        def inner(*args):
            args = args + (arg1,)
            return func(*args)
        return inner
    return outer

final = c1(lambda x,y,z: x+y*z)
print(final(2)(3,4))
```

4. מה תדפיס התוכנית הבאה?

```
vec = [0,0,7,0,1,0,8,9]
sp = {i:val for i,val in enumerate(vec) if val}
print(sp)
```

5. מהי סיבוכיות זמן הריצה של התוכנית הבאה כפונקציה של n?

```
def f(n):
    result = [0]
    for i in range(n//2-18):
        value = max(result)+1
        result.insert(i//2,value) #inserts value at index i//2
    for i in range(len(result)):
        result[i]-=1
    return result
```

6. מהי סיבוכיות זמן הריצה של התוכנית הבאה כפונקציה של n?

```
def f(n):  
    lst = [[1]*n for k in range(n)]  
    for i in range(len(lst)):  
        lst[i] = [sum(lst[i][j:]) for j in range(n)]  
    return lst
```

7. מה תדפיס התוכנית הבאה?

```
def f(start, end, step):  
    print(start, end = " ")  
    if start < end:  
        f(start+step, end, step)  
        print(start, end = " ")  
f(1, 35, 10)
```

8. מה תדפיס התוכנית הבאה?

```
class Backson:  
    __cache = None  
  
    def __init__(self, val):  
        self.__val = val  
        self.__friend = Backson.__cache  
        Backson.__cache = None  
  
        if self.__friend is None:  
            self.__friend = self  
            Backson.__cache = self  
  
    def __str__(self): return self.__val + self.__friend.__val  
  
print(Backson("a"), Backson("b"), Backson("c"), Backson("d"))
```

9. מה תדפיס התוכנית הבאה?

```
def exceptional_func(i):  
    lst = [ValueError(), TypeError(), Exception()]  
    raise lst[i]  
    return i**2  
  
try:  
    for i in range(3):  
        try: print(exceptional_func(i))  
        except ValueError: print("value")  
except: print("smoo")
```

10. מה תדפיס התוכנית הבאה?

```
def foo(x):  
    lst = []  
    do = False  
    for item in x:  
        yield item  
        if do:  
            lst.append(item)  
            do = not do  
    if lst: yield from foo(lst)  
  
print(list(foo(range(4))))
```

חלק ב' – שאלות תכנות (60% – יש לענות רק על 4 מתוך 6 השאלות, כל שאלה 15 נק.)

יש לענות על כל שאלה רק בתוך הקופסה שניתנה בדף. מומלץ להשתמש במחברת הטייטל כדי לגבש את הפתרון ורק כשהוא מושלם להעתיקו לטופס המבחן. אין צורך בהערות או בתיעוד. יש לכתוב קוד בהיר ויעיל.

1. (13 נק') קבוצת שחקני כדורת מיוצגת ע"י רשימה של מספרים. כל מספר מייג את חוזקו של שחקן יחיד בקבוצה. כששתי קבוצות נפגשות, כל שחקן בקבוצה א' משחק נגד כל שחקן בקבוצה ב'. השחקן בעל החוזק הגבוה יותר מנצח תמיד.

כתבו פונקציה המקבלת שתי רשימות של מספרים המייצגות שתי קבוצות וסופרת כמה פעמים שחקנים מקבוצה א' מנצחים את שחקני קבוצה ב'. תיקו (שוויון במספרים) אינו נחשב לניצחון.

הנחות על הקלט: שתי הרשימות אינן ריקות והן ממוינות בסדר עולה. לא ניתן להניח שהרשימות באותו אורך. עליכם לספק פתרון יעיל ככל האפשר. פתרון לא יעיל לא יזכה במלוא הנקודות.

לדוגמא: הפלט עבור $\text{match}([1,8,10],[0,1,9])$ יהיה 6:

1 מנצח את 0 בלבד, 8 מנצח את 0,1, ולבסוף 10 מנצח את כל שלושת יריביו.

```
def match(team1, team2):
```

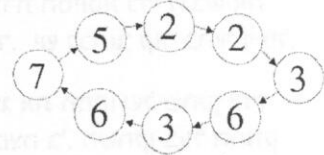
תשובה:	(2 נק') מהי סיבוכיות זמן הריצה של הפתרון עבור שתי רשימות באורך n?
--------	---

```
class Node:
    def __init__(self, data, next=None):
```

```
def __init__(self, data, next=None):
```

```
self.data = data
```

```
self.next = next
```



רשימה מעגלית מתאפיינת בכך שהתא "האחרון" שבה מצביע על התא "הראשון". כיוון שכך, למעשה, אין תא "ראשון" או "אחרון", וכל תא יכול לשמש כ"ראש" הרשימה. באיור מוצגת דוגמה לרשימה מעגלית.

רשימה ריקה מיוצגת, כרגיל, ע"י None. אם יש ברשימה מעגלית איבר אחד בלבד אזי הוא מצביע על עצמו (head = head.next). ואם יש שני איברים אזי הם מצביעים זה על זה.

כתבו פונקציה `reverse(head)` שהופכת את הסדר של כל הקודקודים ברשימה מעגלית (כל קודקוד בסוף מצביע לקודקוד שהצביע עליו ברשימה המקורית).

הנחיות: אסור להשתמש ברקורסיה. מותר להשתמש רק בכמות קבועה של זכרון (למשל אין להעתיק את כל תוכנה של הרשימה המקושרת לתוך רשימה רגילה בדרך לפתרון). אפשר להניח שהרשימה מעגלית תקינה.

```
def reverse(head):
```

```
def reverse(head):
```


4. כתבו פונקציה `brute_search(word, text, num)` המקבלת מחרוזת `word`, מחרוזת `text`, ומספר שלם `num`. הפונקציה בודקת אם המחרוזת `word` מופיעה ב `text` מספר פעמים השווה ל- `num` (או יותר). במקרה זה ערך ההחזרה של הפונקציה יהיה `True`, אחרת `False`. יש לספור מופע של `word` רק אם כל אותיות המילה מופיעות בטקסט ברצף על פי הסדר המקורי (ללא תווים נוספים באמצע). בין הופעות שונות של `word` יכולים להיות תווים אחרים. ההופעות השונות של `word` צריכות לא לחפוף זו לזו.

לדוגמא: אם `word="xyz"`, `text="xyzxyzxyz"` אזי `brute_search(word, text, 1)` יחזיר `True`. ואילו `brute_search(word, text, 2)` יחזיר `False`. באופן דומה, אם `word="xx"`, `text="xxx"` אזי `brute_search(word, text, 2)` יחזיר `True`, ואילו `brute_search(word, text, 1)` יחזיר `False`.

הנחיות: ניתן להניח שהמחרוזות תקינות ואינן ריקות. ניתן להניח ש `num` שלם חיובי ממש. מותר להשתמש בגישה למחרוזות לפי אינדקס, ב-`len`, וגם ב- `slicing` אך לא בפונקציות אחרות של מחרוזות.

```
def brute_search(word, text, num):
```


5. כתבו פונקציה `iterable_to_func(iterable)` המקבלת אובייקט שהוא `iterable` (כלומר ניתן לייצר איטרטורים עליו ללא הגבלה). הפונקציה מחזירה פונקציה `g` כך ש `g(n)` מחזירה את האיבר ה-`n` באיטרטור של `iterable`.

לדוגמא: $g(1)$ תחזיר את האיבר הראשון. $g(3)$ את האיבר השלישי, וכדומה. ניתן להניח שיש באמת איבר H -לאיטרטורים ואין צורך לטפל בשגיאות במידה ואין.
הגבלות: יש לוודא שצריכת הזיכרון של g היא $O(1)$. כלומר לא ניתן לשמור מספר בלתי מוגבל של ערכים.

```
def iterable_to_func(iterable):
```

6. כתבו מחלקה Car. לכל מכונית יש מספר רישוי (מחרוזת). המחלקה תומכת בפעולות הבאות:

- (1) יצירת מכונית עם מספר רישוי נתון: `car1 = Car(license_num)`
- (2) שינוי מספר הרישוי של המכונית, לדוגמא: `car1.change(new_num)`
- (3) החזרת מספר הרישוי של המכונית למספר שקיבלה ביום היווצרה. למשל: `car1.revert()`

בכל אחת מהפעולות הנ"ל יש לזרוק `ValueError` אם כבר קיימת מכונית עם אותו מספר רישוי ולא לבצע את הפעולה. שימו לב: אין להוסיף מבני נתונים חיצוניים למחלקה.

```
class Car:
```

(ניתן להמשיך בעמ' הבא)

[illegible]

בהצלחה!